Software Requirements Specification

Bug Tracking System

Development Team - 2024

Bug Tracking System

Agenda

Project Overview and Objectives

User Roles and Requirements

Functional and Non-Functional Requirements

System Architecture and Database Design

API Specifications and Security

Future Enhancements and Conclusion

Project Overview

A web-based system to track, manage, and resolve software bugs efficiently

Centralized platform for development teams to report and fix issues

Real-time status updates and comprehensive reporting capabilities

Integration support with existing development tools and CI/CD pipelines

Project Objectives

Reduce bug resolution time by 40%

Increase development team productivity

Provide real-time visibility into bug status

Enable data-driven decision making through analytics

Ensure seamless integration with development workflows

User Roles

Administrator: Full system access, user management, configuration

Project Manager: Project oversight, team assignment, reporting

Developer: Bug assignment, status updates, code fixes

Tester: Bug reporting, verification, regression testing

Reporter: Bug submission, status tracking, feedback provision

Stakeholder: Progress monitoring, requirement validation

QA Lead: Test plan coordination, quality metrics review

Support: Customer issue triage, escalation management

Functional Requirements

User authentication and role-based access control

Bug creation with priority, severity, and assignment

Real-time bug status tracking and notifications

Search and filtering capabilities

Reporting and dashboard generation

Integration with version control systems

Non-Functional Requirements

Performance: Handle 1000+ concurrent users with <2s response time

Availability: 99.9% uptime with automated failover

Scalability: Horizontal scaling support for growing teams

Security: End-to-end encryption and compliance with OWASP standards

Usability: Intuitive interface reducing training time by 50%

System Architecture
Frontend: React.js with responsive design
Backend: Node.js with Express framework
Database: PostgreSQL with Redis caching layer
API Layer: RESTful services with GraphQL support
Deployment: Docker containers with Kubernetes orchestration
Monitoring: Prometheus and Grafana for system metrics

Database Design

API Requirements

POST /api/bugs - Create new bug report

GET /api/bugs/{id} - Retrieve bug details

PUT /api/bugs/{id} - Update bug information

GET /api/projects/{id}/bugs - List project bugs

POST /api/auth/login - User authentication

GET /api/reports - Generate analytics reports

Security Features

JWT-based authentication with refresh tokens

Role-based access control (RBAC)

Data encryption at rest and in transit

Input validation and SQL injection prevention

Audit logging for all system activities

Rate limiting to prevent abuse

Future Enhancements

AI-powered bug prediction and assignment

Mobile application for iOS and Android

Advanced analytics with machine learning insights

Third-party tool integrations (Slack, Jira, GitHub)

Automated testing pipeline integration

Multi-language support for global teams

Thank You

Questions and Discussion